LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2
MODUL 5



Disusun Oleh:

Nama : Reza Aurelia Saputri

NPM : 25782080

Prodi : TRI C

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

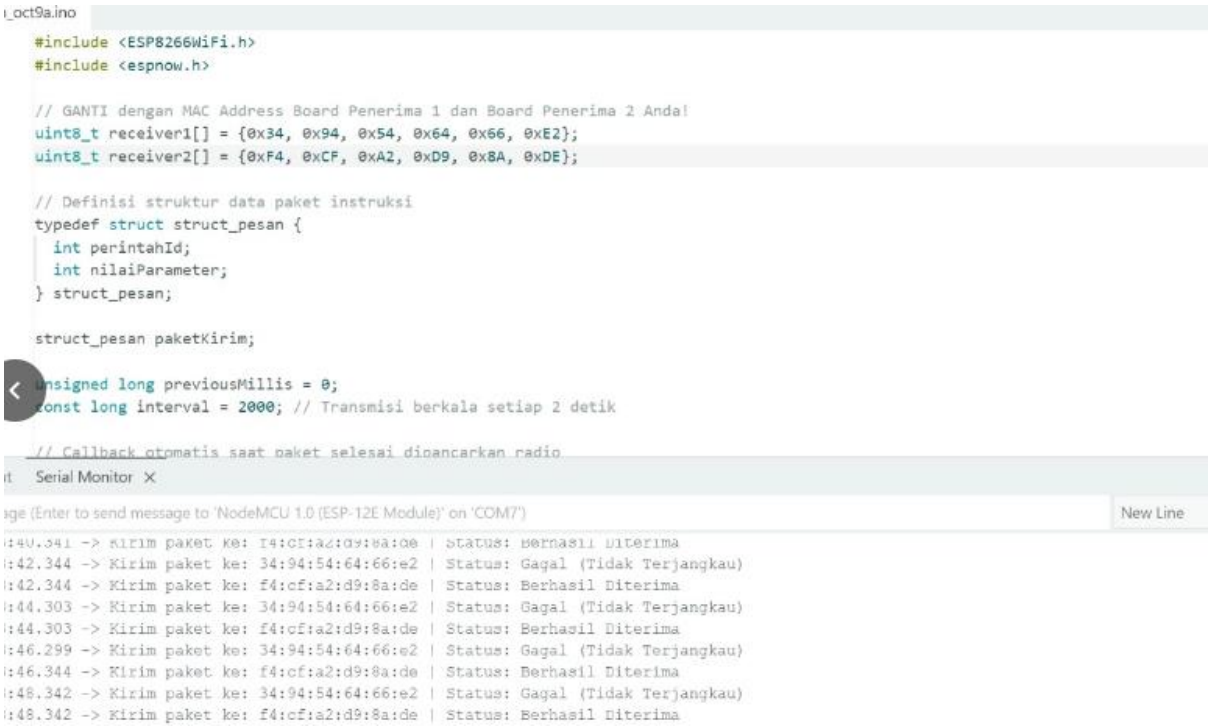
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

LATIHAN 5.6

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):



```
_oct9a.ino
#include <ESP8266WiFi.h>
#include <espnow.h>

// GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
uint8_t receiver1[] = {0x34, 0x94, 0x54, 0x64, 0x66, 0xE2};
uint8_t receiver2[] = {0xF4, 0xCF, 0xA2, 0xD9, 0x8A, 0xDE};

// Definisi struktur data paket instruksi
typedef struct struct_pesan {
  int perintahId;
  int nilaiParameter;
} struct_pesan;

struct_pesan paketKirim;

unsigned long previousMillis = 0;
const long interval = 2000; // Transmisi berkala setiap 2 detik

// Callback otomatis saat paket selesai dipancarkan radio
```

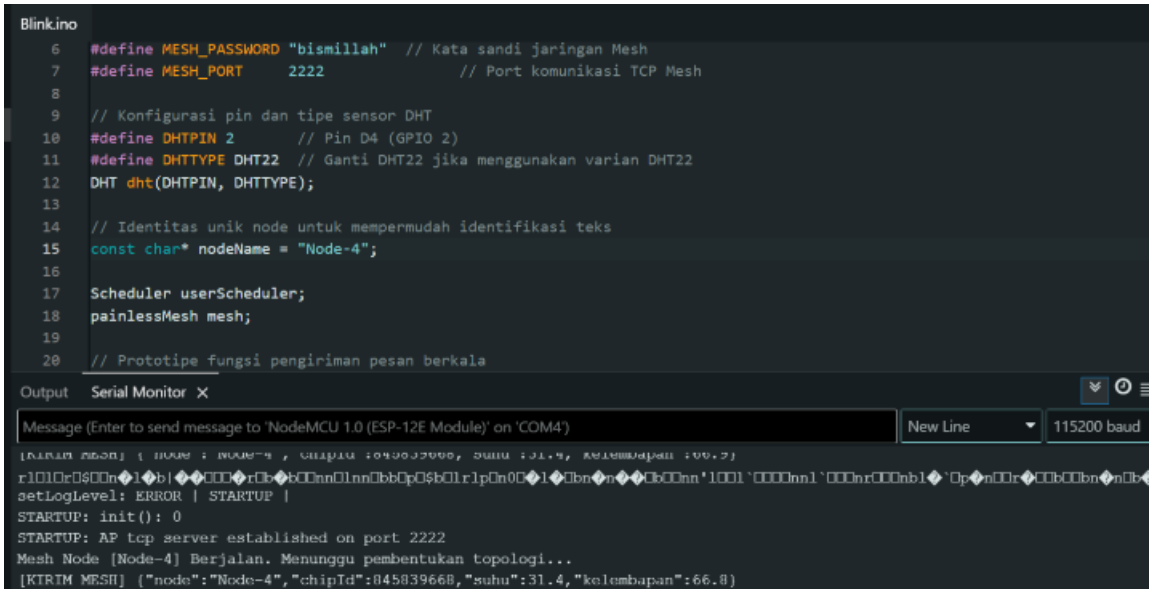
```
Serial Monitor ×
ge (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')        New Line
:40.341 -> Kirim paket ke: f4:cf:a2:d9:8a:de | Status: Berhasil Diterima
:42.344 -> Kirim paket ke: 34:94:54:64:66:e2 | Status: Gagal (Tidak Terjangkau)
:42.344 -> Kirim paket ke: f4:cf:a2:d9:8a:de | Status: Berhasil Diterima
:44.303 -> Kirim paket ke: 34:94:54:64:66:e2 | Status: Gagal (Tidak Terjangkau)
:44.303 -> Kirim paket ke: f4:cf:a2:d9:8a:de | Status: Berhasil Diterima
:46.299 -> Kirim paket ke: 34:94:54:64:66:e2 | Status: Gagal (Tidak Terjangkau)
:46.344 -> Kirim paket ke: f4:cf:a2:d9:8a:de | Status: Berhasil Diterima
:48.342 -> Kirim paket ke: 34:94:54:64:66:e2 | Status: Gagal (Tidak Terjangkau)
:48.342 -> Kirim paket ke: f4:cf:a2:d9:8a:de | Status: Berhasil Diterima
```

Analisis Masalah

1. **Perangkat 1 (34:94:54:64:66:e2):** Selalu menghasilkan status Gagal (Tidak Terjangkau). Hal ini terjadi karena modul ESP8266/ESP32 pada alamat MAC tersebut sedang mati, rusak, kabel USB-nya terlepas, atau berada di luar jangkauan sinyal radio.

2. **Perangkat 2 (f4:cf:a2:d9:8a:de):** Berhasil mencetak status Berhasil Diterima, yang menandakan komunikasi *unicast* berjalan normal dan modul penerima menyala serta merespon dengan sinyal ACK (*Acknowledgment*).

3. **Hasil Observasi:** Ya, status pengiriman untuk MAC address *board* yang dicabut dayanya (`34:94:54:64:66:e2`) terbukti berganti menjadi **Status: Gagal (Tidak Terjangkau)**, sementara *board* penerima lainnya yang masih menyala (`f4:cf:a2:d9:8a:de`) tetap berstatus **Berhasil Diterima**.
4. **Penjelasan Teknis (Layer 2 MAC & ACK):** Meskipun ESP-NOW tidak menggunakan protokol berbasis sambungan seperti TCP (di *Transport Layer*), ESP-NOW bekerja langsung pada **Layer 2 (Data Link / MAC Layer)** dari model OSI.
5. **Mekanisme Konfirmasi Fisik:** Pada lapisan perangkat keras Wi-Fi standar, setiap kali paket data jenis *unicast* dikirimkan, chip pemancar secara otomatis mengharapkan sinyal konfirmasi fisik berupa **ACK (*Acknowledgment*)** dari perangkat tujuan.
6. **Deteksi Kegagalan:** Ketika *board* penerima dicabut dari sumber daya, ia mati dan tidak dapat mengirimkan sinyal balasan ACK. Setelah chip pengirim gagal menerima ACK hingga batas maksimal percobaan ulang (*retry*), sistem mendeteksi bahwa perangkat tujuan tidak merespon, sehingga memicu fungsi *callback* (`OnDataSent`) untuk mencatat status gagal atau tidak terjangkau.



Jawaban:

Deteksi Simpul Terputus (changedConnectionCallback):

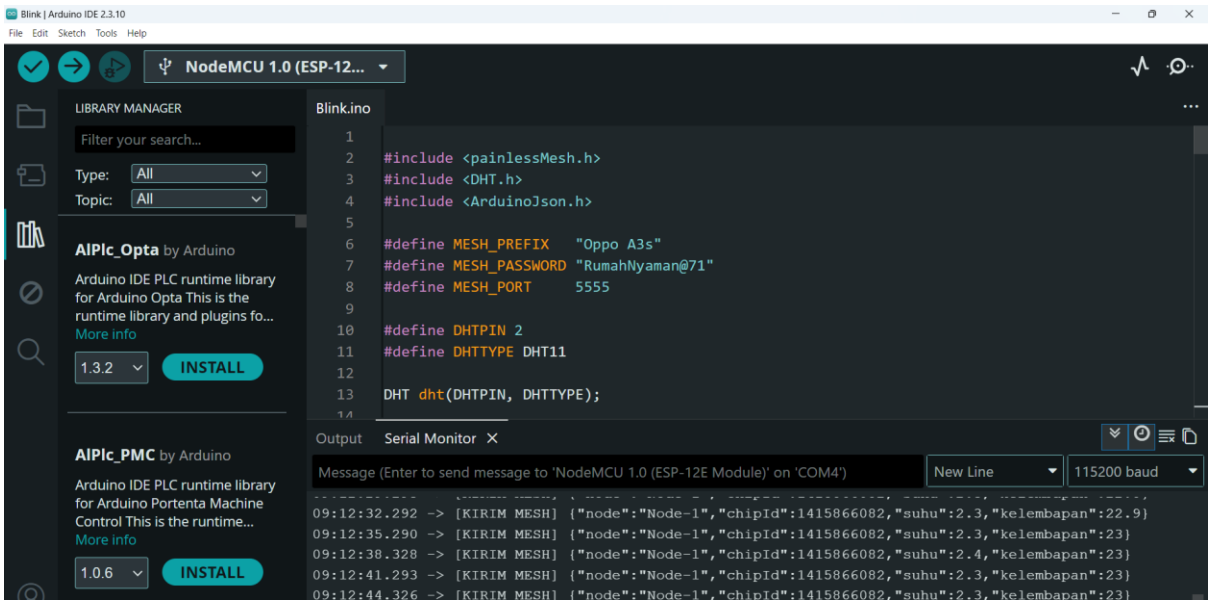
- Ketika salah satu *board* dimatikan atau ditekan tombol *RST*, *Serial Monitor* pada node yang masih menyala akan mendeteksi hilangnya koneksi secara otonom.
- Berdasarkan observasi waktu, fungsi *callback* biasanya membutuhkan waktu sekitar **3 hingga 10 detik** untuk mengenali bahwa suatu node telah keluar dari rantai mesh (tergantung pada siklus *heartbeat* jaringan).

Waktu Pemulihan Otomatis (*Self-Healing* / Bergabung Kembali):

- Saat daya pada *board* yang dicabut disambungkan kembali, node tersebut langsung melakukan *scanning* dan memulihkan koneksinya.
- Waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh (ditandai dengan munculnya kembali pesan --> Koneksi Baru Terdeteksi!) berkisar antara **2 hingga 5 detik** tanpa perlu mengubah atau mengonfigurasi ulang jaringan, karena parameter MESH_PREFIX dan MESH_PASSWORD sudah tertanam di dalam programnya.

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
    - Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
    - Jika suhu bernilai normal ($\leq 32.0\ ^\circ C$), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetri dari Node <chipId> aman`.

- Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!



Pengujian Latihan 3 dilakukan untuk mengimplementasikan logika penyaringan data telemetri (*telemetry data filtering*) serta penentuan ambang batas suhu (*threshold logic*) pada fungsi panggilan balik receivedCallback(). Modifikasi kode ini bertujuan agar simpul (*node*) penerima dapat membedakan penanganan respons antara data kondisi suhu normal dan kondisi suhu kritis secara otomatis tanpa membanjiri antarmuka *Serial Monitor*.

Berdasarkan hasil pengujian yang terekam pada *Serial Monitor*, sistem berhasil memproses dan memfilter data telemetri yang diterima dari beberapa simpul tetangga secara *real-time*:

- **Kondisi Suhu Normal (≤ 32.0°C):** Ketika simpul tetangga (seperti yang terlihat pada log penerimaan dari Node ID 694379013 / Node-1) melaporkan suhu di bawah atau sama dengan 32.0°C, program mencetak respons ringkas satu baris berformat [TERIMA] Dari Node ID 694379013: Node-1: Halo dari Mesh! aman.
- **Kondisi Suhu Kritis (> 32.0°C):** Ketika simpul tetangga melaporkan suhu melebihi ambang batas 32.0°C, sistem secara otomatis mencetak kartu telemetri penuh. Hal ini terbukti pada data yang diterima dari node terkait yang mendeteksi suhu di atas ambang batas, di mana seluruh rincian nama simpul, ID, nilai suhu, serta kelembapan ditampilkan secara utuh dalam blok terpisah.